

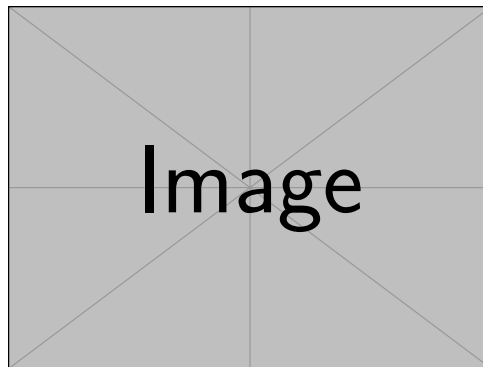
MagistralaCAN4

Wysokowydajny sniffer CAN z uczeniem asocjacyjnym i skryptami Lua

Dokumentacja techniczna – wersja rozszerzona

5 maja 2026

Narzędzie do analizy i diagnostyki magistrali CAN/CAN FD



Spis treści

1	Wprowadzenie	3
2	Architektura systemu	3
2.1	Ogólny schemat blokowy	3
2.2	Warstwy systemu	4
2.3	Przepływ danych	4
3	Opis modułów	5
3.1	CanSniffer	5
3.2	CanFrameModel	5
3.3	AssociativeLearner	5
3.4	GpuCorrelator	6
3.5	LuaScriptEngine	6
3.6	DbcParser i CanDashboard	6
3.7	OfflineAnalyze	6
3.8	FrameDetailWidget	6
3.9	CandidateModel	6
3.10	Logger	7
4	Interfejs użytkownika	7
4.1	Pasek narzędzi	7
4.2	Zakładki	7
4.3	Skróty klawiszowe	7
4.4	Powiadomienia systemowe	7
5	Uczenie asocjacyjne – przewodnik	7
5.1	Przygotowanie danych	7
5.2	Analiza zdarzeń i tła	8
5.3	Korelacje, MI i MIC	8
5.4	Predykcja i sekwencje	8
5.5	Klastrowanie i PCA	8
5.6	Eksport modeli i reguł Lua	8
6	Silnik Lua	8
7	Eksport i import	9
8	Plany rozwoju	9

1 Wprowadzenie

MagistralaCAN4 jest zaawansowanym, wielowątkowym narzędziem dla inżynierów pracujących z magistralami CAN i CAN FD. Powstało z myślą o szybkiej identyfikacji zależności pomiędzy zdarzeniami fizycznymi (np. naciśnięcie hamulca) a ruchem na magistrali, przy jednoczesnym zapewnieniu maksymalnej wydajności dzięki wykorzystaniu GPU oraz intuicyjnego interfejsu użytkownika.

Główne możliwości:

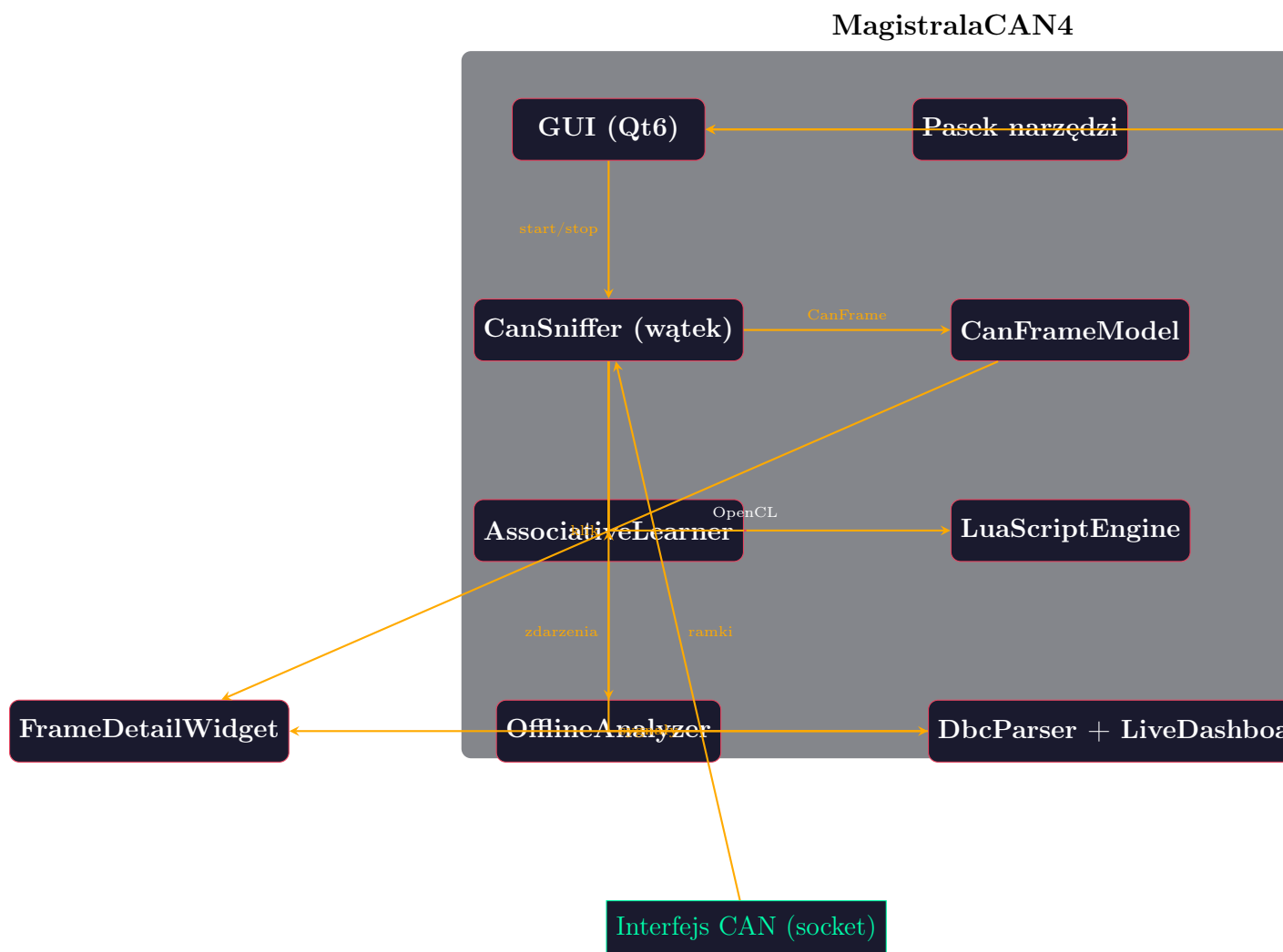
- **Przechwytywanie ramek** CAN 2.0 i CAN FD z interfejsów socketCAN z nadpisywaniem lub dopisywaniem.
- **Uczenie asocjacyjne** oparte na analizie okien czasowych, kontraście ze zdarzeniami tła, korelacjach liniowych i nieliniowych (Pearson, Mutual Information, MIC), regresji liniowej, łańcuchach Markowa, k-średnich, PCA oraz automatycznej detekcji zdarzeń.
- **Silnik skryptowy Lua** umożliwiający automatyzację reakcji na ramki (wysyłanie, logowanie) poprzez API `sendFrame`, `log`, `getTick`.
- **Obsługa plików DBC** – parsowanie, wyświetlanie nazwanych sygnałów w widoku szczegółów ramki oraz dashboard na żywo z aktualnymi wartościami.
- **Analiza offline** – wczytywanie plików `candump` z odtwarzaniem z oryginalnymi znacznikami czasu, trybem krokowym i pełną integracją z modułem uczenia.
- **Eksport** do formatu `candump`, modeli asocjacyjnych (JSON) oraz automatyczne generowanie skryptów Lua na podstawie wyuczonych reguł.
- **Interaktywny edytor bajtów** w widoku ramki z możliwością wysyłania zmodyfikowanych ramek.
- **Powiadomienia systemowe** (tray) oraz globalne skróty klawiszowe (Ctrl+Shift+E dla zdarzenia).

Projekt jest rozwijany w C++17 z wykorzystaniem Qt6, OpenCL oraz Lua. Architektura oparta jest na osobnych wątkach dla akwizycji i interfejsu, a obliczenia mogą być akcelerowane przez GPU. Kod źródłowy podzielony jest na czytelne moduły, co ułatwia dalszy rozwój.

2 Architektura systemu

2.1 Ogólny schemat blokowy

Poniższy diagram (rys. 1) przedstawia główne bloki funkcjonalne aplikacji wraz z przepływem danych.



Rysunek 1: Ogólny schemat blokowy systemu MagistralaCAN4.

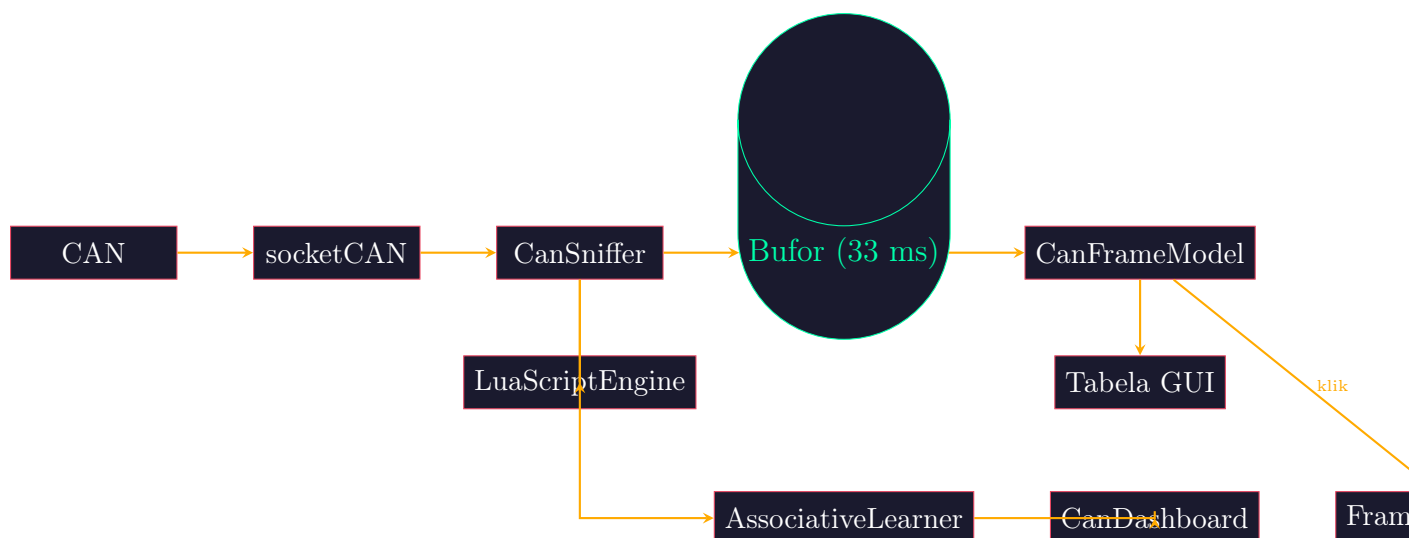
2.2 Warstwy systemu

Aplikację można podzielić na trzy logiczne warstwy:

1. **Warstwa interfejsu użytkownika** – odpowiada za prezentację danych (tabele, wykresy, dashboard) oraz interakcję z operatorem. Wykorzystuje Qt6 Widgets, Charts oraz QSystemTrayIcon dla powiadomień.
2. **Warstwa logiki** – zawiera moduły analityczne (uczenie asocjacyjne, statystyka, Lua) oraz zarządzanie danymi. Tutaj wykonywane są obliczenia, również z wykorzystaniem GPU (OpenCL) i wielowątkowości CPU (QtConcurrent).
3. **Warstwa komunikacji** – odpowiada za odczyt i zapis ramek CAN poprzez gniazda socketCAN. Uruchamiana jest w osobnym wątku, aby nie blokować GUI.

2.3 Przepływ danych

Rysunek 2 przedstawia szczegółowy przepływ ramek od magistrali do poszczególnych konsumentów.



Rysunek 2: Przepływ danych w systemie.

Ramki z magistrali są odczytywane w wątku CanSniffer i emitowane przez sygnał `newFrame`. Główny wątek buforuje je przez 33 ms, po czym wsadowo aktualizuje model tabeli. Równoległe te same ramki trafiają do modułów asocjacyjnego, Lua oraz dashboardu.

3 Opis modułów

3.1 CanSniffer

Klasa odpowiada za otwarcie gniazda CAN RAW (z obsługą CAN FD) oraz odczyt ramek w pętli nieskończonej. Uruchamiana w osobnym wątku przez `QtConcurrent::run`. Ramki są parsowane do struktury `CanFrame` i emitowane jako sygnał. Obsługuje również wysyłanie ramek (`writeFrame`) wykorzystywane przez Lua i edytor.

3.2 CanFrameModel

Model tabeli dziedziczący po `QAbstractTableModel`. Przechowuje listę ramek (z opcjonalnym nadpisywaniem wg ID) i aktualizuje widok w trybie wsadowym co 33 ms, co zapewnia wysoką wydajność przy dużym obciążeniu.

3.3 AssociativeLearner

Centralny moduł analityczny. Udostępnia szereg analiz:

- **Lista kandydatów** – identyfikatory CAN pojawiające się we wszystkich zdarzeniach; ranking na podstawie podobieństwa wektorów cech.
- **Korelacja wartość–bajt** – współczynniki Pearsona między wprowadzaną zmienną (np. temperatura) a bajtami ramek.
- **Sekwencje** – najczęstsze bigramy i trigramy ID w oknach zdarzeń.
- **Korelacja międzybajtowa** – zależności między bajtami różnych identyfikatorów.
- **Mutual Information (MI)** – nieliniowa miara zależności.

- **MIC (Maximal Information Coefficient)** – znormalizowana (0–1) miara zdolna wykryć dowolne zależności.
- **Predykcja wartości** – regresja liniowa dla silnie skorelowanych bajtów, bieżąca prognoza zmiennej.
- **Predykcja sekwencji (Markov)** – na podstawie historii ramek przewiduje najbardziej prawdopodobne następne ID.
- **Detekcja anomalii** – buduje model normalny i sygnalizuje odchylenia.
- **Auto-detekcja zdarzeń** – monitoruje gradient zmiennej i korelacje, automatycznie rejestruje zdarzenia.
- **Klastrowanie (k-średnich)** – grupuje okna ramek w stany magistrali.
- **PCA + k-średnich** – redukcja wymiarów do 2D, wizualizacja klastrów.
- **Macierz korelacji zmiennych** – heatmapa zależności między wszystkimi zmiennymi.
- **Eksport/import modeli** – zapis i odczyt wyuczonych parametrów (JSON).
- **Generator skryptów Lua** – tworzy reguły na podstawie najlepszych kandydatów.

3.4 GpuCorrelator

Klasa wykorzystująca OpenCL do szybkiego obliczania macierzy korelacji (iloczyn skalarny). W przypadku braku GPU przełącza się na wielowątkowy CPU.

3.5 LuaScriptEngine

Silnik Lua z predefiniowanymi funkcjami: `sendFrame(id, data_table)`, `log("tekst")`, `getTick()`. Wczytuje pliki `.lua` zawierające funkcję `onFrame(id, data, timestamp)`.

3.6 DbcParser i CanDashboard

Parser plików DBC ekstrahuje definicje wiadomości i sygnałów. Dashboard wyświetla aktualne wartości sygnałów w czasie rzeczywistym na podstawie ostatnich ramek.

3.7 OfflineAnalyzer

Umożliwia wczytanie pliku `candump`, odtwarzanie z oryginalnymi odstępami czasu (regulowana prędkość), tryb krokowy (Enter) oraz integrację z modulem asocjacyjnym.

3.8 FrameDetailWidget

Widok szczegółów ramki: identyfikator, DLC, czas, siatka 8×8 bitów z podświetlaniem zmian. Po wczytaniu DBC wyświetla nazwy sygnałów. Tryb edycji pozwala zmieniać bajty i bity, a następnie wysłać zmodyfikowaną ramkę.

3.9 CandidateModel

Prosty model listy kandydatów (CAN ID, opis, pewność, wystąpienia) używany przez `AssociativeLearner`.

3.10 Logger

Zapisuje zdarzenia (z timestampem) do `/magistrala_can4.log` oraz wypisuje na konsolę.

4 Interfejs użytkownika

4.1 Pasek narzędzi

Zawiera:

- ComboBox z listą dostępnych interfejsów CAN (odświeżanie przyciskiem),
- Przycisk Start/Stop,
- Checkbox nadpisywania ramek,
- Etykieta statusu,
- Akcje: Wyczyść, Eksportuj candump, Wczytaj Lua, Wczytaj DBC.

4.2 Zakładki

1. **Ruch CAN** – tabela z kolumnami: CAN ID, Typ (STD/EXT), RTR, DLC, Dane (hex), Czas.
2. **Uczenie asocjacyjne** – przewijalny panel z polami do zarządzania zmiennymi i przyciskami akcji oraz licznymi tabelami wyników i wykresami.
3. **Szczegóły ramki** – interaktywna siatka bajtów/bitów.
4. **Analiza offline** – kontrolki do wczytywania i odtwarzania plików.
5. **Dashboard CAN** – panele z aktualnymi wartościami sygnałów DBC.

4.3 Skróty klawiszowe

- Ctrl+Shift+E – zarejestruj zdarzenie,
- Ctrl+Shift+D – zarejestruj brak zdarzenia,
- Enter – w trybie offline odtwarza następną ramkę.

4.4 Powiadomienia systemowe

Aplikacja wyświetla dymki systemowe przy zdarzeniach asocjacyjnych i anomaliach. Ikona w tray'u umożliwia szybki dostęp.

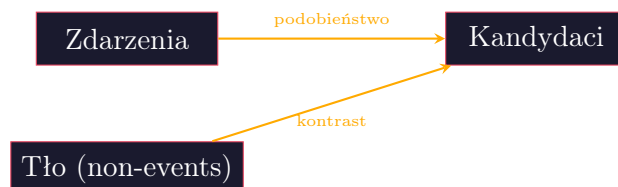
5 Uczenie asocjacyjne – przewodnik

5.1 Przygotowanie danych

Należy wybrać (lub dodać) zmienną liczbową (np. temperatura, obroty) i wprowadzać jej wartości w momencie obserwacji. Program przechowuje wartości wraz ze średnimi bajtami ramek z okna czasowego (± 500 ms, adaptacyjne).

5.2 Analiza zdarzeń i tła

Kliknięcie **Zarejestruj zdarzenie** zapamiętuje bieżące okno ramek jako zdarzenie. Kliknięcie **Brak zdarzenia** zapisuje okno jako tło. Algorytm porównuje cechy ID pomiędzy zdarzeniami a tłem – im bardziej ID przypomina tło, tym jego punktacja jest obniżana.



Rysunek 3: Wpływ zdarzeń i tła na listę kandydatów.

5.3 Korelacje, MI i MIC

Po zebraniu co najmniej 3 obserwacji można obliczyć:

- **Korelację Pearsona** – standardową miarę liniową.
- **Mutual Information** – opartą na entropii, wykrywa nieliniowości.
- **MIC** – znormalizowaną miarę odporną na typ zależności.

Wyniki prezentowane są w tabelach z kolorowaniem (zielony – silna zależność).

5.4 Predykcja i sekwencje

Regresja liniowa tworzy model $y = a \cdot x + b$ dla każdej pary (ID, bajt) o wysokiej korelacji. Co 500 ms aktualizowana jest prognoza wartości zmiennej. Łańcuch Markowa zlicza przejścia między ID i wyświetla najbardziej prawdopodobny następny identyfikator.

5.5 Klastrowanie i PCA

K-średnich (K=3 domyślnie, z opcją automatycznego doboru metodą łokcia) grupuje okna czasowe w stany magistrali. PCA redukuje wymiar do 2D i umożliwia wizualizację (rys. ??).

5.6 Eksport modeli i reguł Lua

Wyuczone modele (parametry regresji, łańcuch Markowa, model normalny) można zapisać do JSON i później wczytać. Przycisk **Generuj skrypt Lua** tworzy plik .lua z regułami if na podstawie najlepszych kandydatów.

6 Silnik Lua

Skrypty Lua mogą być wczytywane w trakcie pracy. Przykład:

```

1 function onFrame(id, data, timestamp)
2     if id == 0x100 then
3         log(string.format("ID=0x%X, byte0=%d", id, data[1]))
  
```



```
4     end
5     if data[2] > 200 then
6         sendFrame(0x200, {0x01, 0x02})
7     end
8 end
```

Listing 1: example.lua

API:

- `sendFrame(id, table)` – wysyła ramkę; zwraca `true` / `false`.
- `log(msg)` – wypisuje komunikat do logu i konsoli.
- `getTick()` – zwraca czas od uruchomienia (ms).

7 Eksport i import

- **candump** – eksport ramek do pliku tekstowego w formacie kompatybilnym z narzędziem `candump`.
- **Modele asocjacyjne** – zapis/odczyt parametrów regresji, łańcucha Markowa, modelu normalnego (JSON).
- **Generator Lua** – automatyczne tworzenie skryptów na podstawie wyuczonych reguł.

8 Plany rozwoju

1. **Integracja z chmurą** – przesyłanie logów i modeli do AWS IoT / Azure, zdalny podgląd dashboardu.
2. **Interfejs webowy** – lekki serwer HTTP w aplikacji umożliwiający podgląd na żywo z przeglądarki.
3. **Zaawansowane modele ML** – LSTM, Transformer do przewidywania sekwencji i anomalii.
4. **Wsparcie dla CAN XL** – obsługa ramek o długości do 2048 bajtów.
5. **Edytor DBC** – graficzne tworzenie i modyfikacja plików DBC.
6. **Nagrywanie i odtwarzanie z kompresją** – zapis sesji w formacie binarnym.
7. **Więcej skrótów klawiszowych** – globalne skróty dla innych funkcji (np. eksport).
8. **Automatyczne raporty HTML** – generowanie podsumowania analizy w formacie HTML z wykresami.

Dodatek A – Schemat potoku uczenia



Rysunek 4: Potok analizy: od ramek do wyników.

Dodatek B – Struktura pliku DBC (uproszczona)

```
1 B0_ 1234 ExampleMsg: 8 Vector__XXX
2 SG_ EngineSpeed : 0|16@1+ (0.5,0) [0|8000] "rpm" Vector__XXX
3 SG_ Temperature : 16|8@1+ (1,-40) [-40|210] "degC" Vector__XXX
```

Listing 2: Fragment pliku DBC